นายนฤเศรษฐ์ อภิลักขิตพงศ์	**673380044-4**	QA/Test
นายวัชรพล ดวงกองเงิน	**673380290-9**	Network Architects
นาย คณิศร มาประจักษ์	**673380031-3**	IaC/DevOps
นายสิรภัทร ลีล้าน	**673380067-2**	Network Architects
นาย ณัฐกร รุ่งฟ้า	**673380512-7**	IaC/DevOps

Lab 4 using your specific repository, layout, and networking assumptions. This version **treats Local subnets as private networks** and **treats the remote server / inter-site network as a 10.10.x.x "internet/public network"** for the purposes of NAT and routing logic. It also assumes the repository hierarchy:

```
None
automation/
├── mockup-infra/
└── week02-stateless-stateful/
    └── phase1-mockup/
```

We will produce:

✅ A revised **high-fidelity lab plan** ✅ **Detailed addressing & topologies** ✅ **Cisco-themed network section** ✅ **IaC / VS Code section** ✅ **Troubleshooting and engineering reflection** ✅ A complete **README.md** ready to be committed to the repo

---

🧪 LAB 4 Simulated Internet (10.10.0.0/16) & Private LANs with Stateful vs Stateless Services

---

🎯 LAB GOALS

1. **Network Setup:** Create two local labs (A & B) connected via a simulated "public/internet" network (10.10.0.0/16)
2. **NAT + Routing:** Configure static routes and NAT so local traffic reaches remote services
3. **Application Deployment:** Use `mockup-infra` from your repository to deploy stateless and stateful APIs
4. **Testing:** Compare how sessions behave locally vs across the simulated internet

---

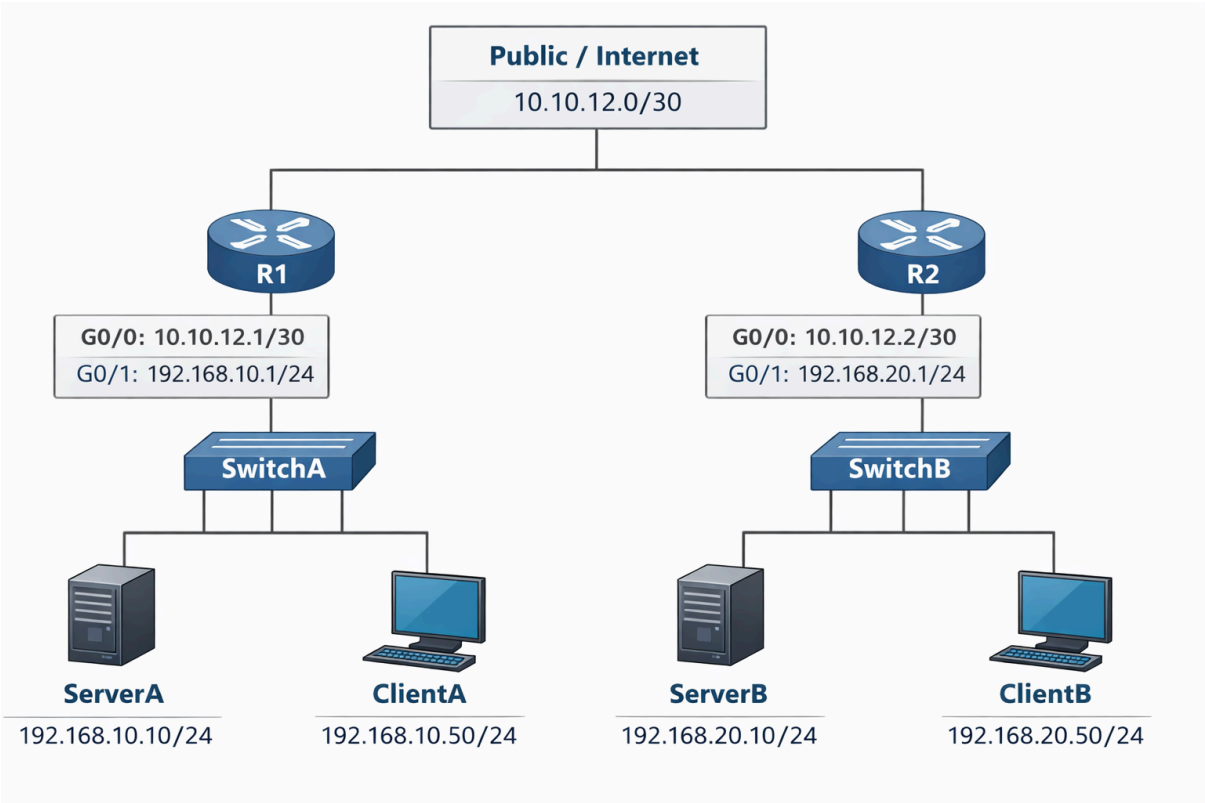
👥 TEAM BREAKDOWN (5–6 students)

Role	Count	Focus
Network Architects	2	Router configs, Cisco CLI
IaC/DevOps	2	VS Code, mockup-infra
QA/Test	1–2	Server/Client scripting, session behavior

🗺️ UPDATED NETWORK TOPOLOGY

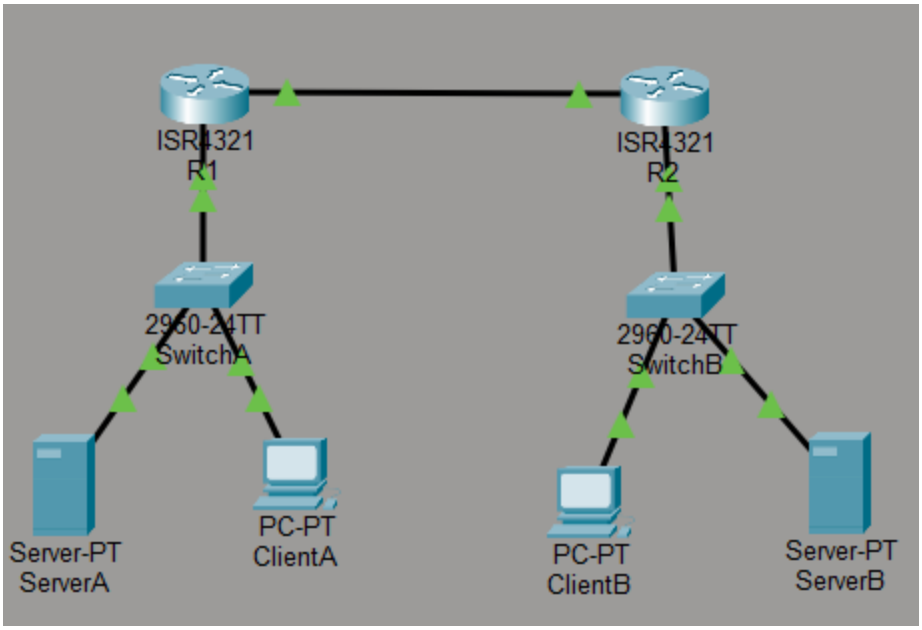
```python
                10.10.12.0/30 “Public/Internet”
             ┌──────────────────────────┐
             |                          |
         R1-G0/0 (10.10.12.1)      R2-G0/0 (10.10.12.2)
             |                          |
   Private LAN A (192.168.10.0/24)     Private LAN B (192.168.20.0/24)
             |                          |
         SwitchA                      SwitchB
             |                          |
 ServerA (mockup)           ServerB (mockup)
 ClientA                        ClientB
```

We treat 10.10.12.0/30 as a **simulated internet** for the purposes of NAT + public reachability.



📍 ADDRESS TABLE

Component	Interface	IP	Mask	Role
R1	G0/0	10.10.12.1	/30	Public
R1	G0/1	192.168.10.1	/24	LAN A Gateway
ServerA	eth0	192.168.10.10	/24	Mockup infra A
ClientA	eth0	192.168.10.50	/24	Tester
R2	G0/0	10.10.12.2	/30	Public
R2	G0/1	192.168.20.1	/24	LAN B Gateway
ServerB	eth0	192.168.20.10	/24	Mockup infra B
ClientB	eth0	192.168.20.50	/24	Tester



🔧 PART 1 — CISCO-STYLE NETWORK CONFIGURATION

We will configure static routing + NAT to allow cross-site reachability.

🧱 R1 CONFIGURATION (LAN A)

```
None
enable
conf t

hostname R1

interface g0/0
```

```
 ip address 10.10.12.1 255.255.255.252
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.10.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.20.0 255.255.255.0 10.10.12.2

access-list 10 permit 192.168.10.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#hostname R1
R1(config)#interface g0/0
R1(config-if)# ip address 10.10.12.1 255.255.255.252
R1(config-if)#ip nat outside
R1(config-if)#no shutdown

R1(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

R1(config-if)#interface g0/1
R1(config-if)#ip address 192.168.10.1 255.255.255.0
R1(config-if)#ip nat inside
R1(config-if)#no shutdown

R1(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up

R1(config-if)#ip route 192.168.20.0 255.255.255.0 10.10.12.2
R1(config)#access-list 10 permit 192.168.10.0 0.0.0.255
R1(config)#ip nat inside source list 10 interface g0/0 overload
R1(config)#end
R1#
%SYS-5-CONFIG_I: Configured from console by console

R1#write memory
Building configuration...
[OK]
R1#
```

🧱 R2 CONFIGURATION (LAN B)

```
None
enable
conf t
hostname R2
interface g0/0
 ip address 10.10.12.2 255.255.255.252
 ip nat outside
 no shutdown
interface g0/1
 ip address 192.168.20.1 255.255.255.0
 ip nat inside
 no shutdown
ip route 192.168.10.0 255.255.255.0 10.10.12.1
access-list 10 permit 192.168.20.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload
end
write memory
```

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#hostname R2
R2(config)#interface g0/0
R2(config-if)# ip address 10.10.12.2 255.255.255.252
R2(config-if)# ip nat outside
R2(config-if)# no shutdown

R2(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

R2(config-if)#interface g0/1
R2(config-if)# ip address 192.168.20.1 255.255.255.0
R2(config-if)# ip nat inside
R2(config-if)# no shutdown

R2(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up

R2(config-if)#ip route 192.168.10.0 255.255.255.0 10.10.12.1
R2(config)#access-list 10 permit 192.168.20.0 0.0.0.255
R2(config)#ip nat inside source list 10 interface g0/0 overload
R2(config)#end
R2#
%SYS-5-CONFIG_I: Configured from console by console

R2#write memory
Building configuration...
[OK]
R2#
```

🧪 ROUTING & NAT VERIFICATION

On R1 & R2:

```
None
show ip route
show ip nat translations
```

```
R1#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C       10.10.12.0/30 is directly connected, GigabitEthernet0/0
L       10.10.12.1/32 is directly connected, GigabitEthernet0/0
     192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.10.0/24 is directly connected, GigabitEthernet0/1
L       192.168.10.1/32 is directly connected, GigabitEthernet0/1
S    192.168.20.0/24 [1/0] via 10.10.12.2
R1#show ip nat translations
Pro  Inside global     Inside local       Outside local      Outside global
icmp 10.10.12.1:1      192.168.10.50:1    192.168.20.10:1    192.168.20.10:1
icmp 10.10.12.1:2      192.168.10.50:2    192.168.20.10:2    192.168.20.10:2
icmp 10.10.12.1:3      192.168.10.50:3    192.168.20.10:3    192.168.20.10:3
icmp 10.10.12.1:4      192.168.10.50:4    192.168.20.10:4    192.168.20.10:4
```

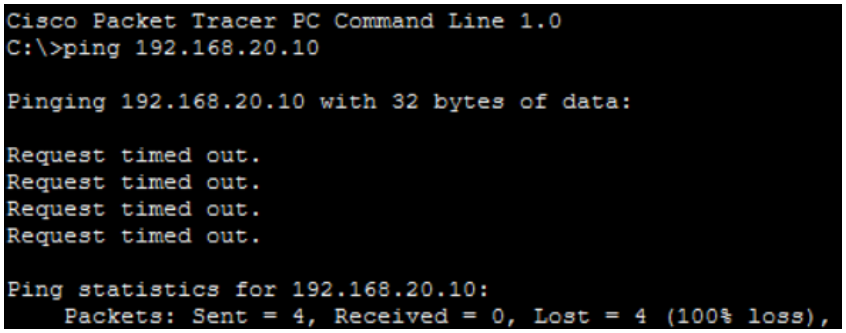
```
R2#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C       10.10.12.0/30 is directly connected, GigabitEthernet0/0
L       10.10.12.2/32 is directly connected, GigabitEthernet0/0
S    192.168.10.0/24 [1/0] via 10.10.12.1
     192.168.20.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.20.0/24 is directly connected, GigabitEthernet0/1
L       192.168.20.1/32 is directly connected, GigabitEthernet0/1
```

From ClientA:

```
None
ping 192.168.20.10  ← Remote server reachable
```



💻 PART 2 — IaC + MOCKUP-INFRA DEPLOYMENT (mockup-infra/README.md)

We will now set up mockup infra using the paths you specified:

```
None
automation/mockup-infra
automation/week02-stateless-stateful/phase1-mockup
```

🛠️ CLONE REPO

```
Shell
git clone https://github.com/boonsup/automation.git
cd automation
```

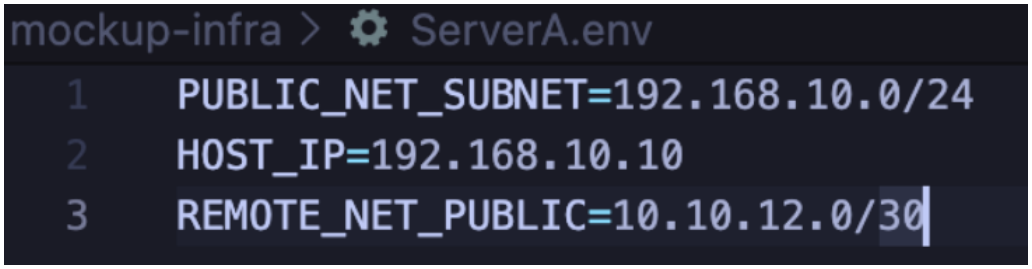
🧰 VS CODE — OPEN PROJECT

In VS Code, open the `automation` folder.

📄 CREATE SERVER-SPECIFIC ENV FILES

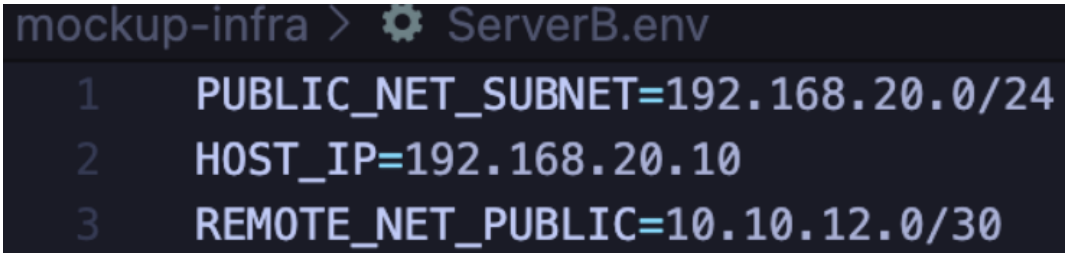
ServerA (`ServerA.env`)

```
None
# Private LAN A
PUBLIC_NET_SUBNET=192.168.10.0/24
HOST_IP=192.168.10.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```



ServerB (`ServerB.env`)

```
None
# Private LAN B
PUBLIC_NET_SUBNET=192.168.20.0/24
HOST_IP=192.168.20.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```



🚀 DEPLOY MOCKUP-INFRA

PowerShell (Windows Admin)

```
None
cd mockup-infra
python manage.py init
python manage.py deploy --profile week02-stateless-stateful
```

Bash (Linux)

```
Shell
cd mockup-infra
chmod +x scripts/*.sh
./scripts/init-env.sh
python manage.py deploy --profile week02-stateless-stateful
```

```
🔧 Initializing Mockup Infrastructure...
==================================================

🔐 [Layer 5/6 - Session] Generating TLS Certificates...
C:\Users\USER\Desktop\KKU work\Y2 sem 2\Network\LAB04_state\automation\mockup-infra\manage.py:118: DeprecationWarning:
 datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to
 represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  .not_valid_before(datetime.utcnow())
C:\Users\USER\Desktop\KKU work\Y2 sem 2\Network\LAB04_state\automation\mockup-infra\manage.py:119: DeprecationWarning:
 datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to
 represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  .not_valid_after(datetime.utcnow() + timedelta(days=365))
✅ Certificates generated successfully:
   - Private Key: C:\Users\USER\Desktop\KKU work\Y2 sem 2\Network\LAB04_state\automation\mockup-infra\certs\server.key
   - Certificate: C:\Users\USER\Desktop\KKU work\Y2 sem 2\Network\LAB04_state\automation\mockup-infra\certs\server.crt
   - Common Name: api.mockup.test
   - Subject Alt Names: localhost, api.mockup.test, 127.0.0.1, 172.18.0.1, 172.19.0.1

🌐 [Layer 3 - Network] Testing Isolation...
🔧 Executing: podman network inspect private_net 2>/dev/null || echo '{}'
The system cannot find the path specified.
'{}'
⚠️ Private network not found or not isolated. Creating networks...
🔧 Executing: podman network create public_net 2>/dev/null || true
The system cannot find the path specified.
'true' is not recognized as an internal or external command,
operable program or batch file.
🔧 Executing: podman network create --internal private_net 2>/dev/null || true
The system cannot find the path specified.
'true' is not recognized as an internal or external command,
operable program or batch file.
✅ Networks created successfully
```

🟡 VERIFY CONTAINERS

```
None
podman ps
```

Should show:

Container	Port
stateless-api	3000
stateful-api	3001

🧪 PART 3 — TESTING STATELESS vs STATEFUL (week03-stateless-stateful/phase1-mockup/README.md)

⚫ LOCAL TESTS (Private LAN)

From ClientA:

```
Shell
curl http://192.168.10.10:3000/info
```

→ Stateless

From ClientA:

```
Shell
curl -X POST http://192.168.10.10:3001/session
```

Store the returned `Session-ID`, then:

```shell
curl http://192.168.10.10:3001/data -H "Session-ID: <id>"
```

การทดสอบ Stateless และ Stateful Service ใน Private LAN

การทดสอบนี้เป็นการตรวจสอบการทำงานของบริการแบบ **Stateless** และ **Stateful** ภายในเครือข่าย Private LAN โดยส่ง HTTP request จากเครื่อง **ClientA** ไปยัง **ServerA (192.168.10.10)**

1. Stateless Service
ใช้คำสั่ง
`curl http://192.168.10.10:3000/info`
เพื่อส่ง HTTP GET request ไปยังเซิร์ฟเวอร์และรับข้อมูลสถานะในรูปแบบ JSON เช่น ชื่อเซิร์ฟเวอร์ เวลา และค่าที่สุ่มขึ้นมาใหม่ในแต่ละครั้ง โดยบริการแบบ Stateless จะไม่เก็บข้อมูลของ request ก่อนหน้า ทำให้แต่ละ request ถูกประมวลผลแยกจากกัน

2. Stateful Service
เริ่มจากสร้าง Session ด้วยคำสั่ง
`curl -X POST http://192.168.10.10:3001/session`
เซิร์ฟเวอร์จะสร้าง Session และส่ง **Session-ID** กลับมา จากนั้นใช้คำสั่ง
`curl http://192.168.10.10:3001/data -H "Session-ID: <id>"`
เพื่อส่ง request พร้อม Session-ID ให้เซิร์ฟเวอร์ตรวจสอบและดึงข้อมูลของ session นั้น แสดงให้เห็นว่าระบบมีการเก็บสถานะของผู้ใช้งาน

สรุป
Stateless ไม่เก็บข้อมูลของ request ก่อนหน้า ส่วน Stateful จะเก็บสถานะของผู้ใช้งานผ่าน Session และใช้ Session-ID เพื่อจดจำการติดต่อครั้งถัดไป.

🔵 REMOTE TESTS (Via Simulated Internet)

From ClientA:

```none
curl http://10.10.12.2:3000/info
```

That will target **ServerB stateless** — NAT + routing should deliver

---

Session Test:

```
None
curl -X POST http://10.10.12.2:3001/session
```

→ ServerB returns a session ID

Then:

```
None
curl http://10.10.12.2:3001/data -H "Session-ID: <id>"
```

Check Session Persists Across NAT

การทดสอบนี้เป็นการตรวจสอบการทำงานของระบบเมื่อมีการเชื่อมต่อข้ามเครือข่าย โดยส่ง HTTP request จาก **ClientA** ไปยัง **ServerB** ผ่าน IP ของ **Router R2 (10.10.12.2)** ซึ่งทำหน้าที่แปลงที่อยู่ IP ด้วย **NAT** ก่อนส่งต่อไปยังเซิร์ฟเวอร์ภายในเครือข่าย

1. Stateless Service ผ่านเครือข่ายภายนอก
ใช้คำสั่ง
`curl http://10.10.12.2:3000/info`
เพื่อส่ง HTTP GET request ผ่าน Router ไปยัง ServerB ซึ่งจะตอบกลับข้อมูลสถานะของเซิร์ฟเวอร์ในรูปแบบ JSON แสดงให้เห็นว่าบริการแบบ **Stateless** ยังทำงานได้ตามปกติแม้ต้องผ่าน Router และ NAT

2. Stateful Service ผ่านเครือข่ายภายนอก
เริ่มจากสร้าง session ด้วยคำสั่ง
`curl -X POST http://10.10.12.2:3001/session`
เซิร์ฟเวอร์จะสร้าง **Session-ID** และส่งกลับมา จากนั้นใช้คำสั่ง
`curl http://10.10.12.2:3001/data -H "Session-ID: <id>"`
เพื่อส่ง request พร้อม Session-ID ให้เซิร์ฟเวอร์ตรวจสอบและดึงข้อมูลของ session นั้น แสดงให้เห็นว่าระบบสามารถจดจำสถานะของผู้ใช้ได้แม้การเชื่อมต่อจะผ่าน NAT

สรุป
ClientA สามารถติดต่อ ServerB ผ่าน Router และ NAT ได้สำเร็จ โดยบริการ **Stateless** ทำงานแบบอิสระต่อแต่ละ request ส่วน **Stateful** สามารถสร้างและรักษา session ของผู้ใช้งานได้แม้จะสื่อสารข้ามเครือข่าย.

⚠️ TROUBLESHOOTING

❗ Cannot Reach Remote

- Gateway misconfiguration
- NAT ACL incorrect
- Wrong default gateway on clients

Check:

```
None
show ip route
ping 10.10.12.2 from R1
```

❗ Sessions Lose State

- Missing header
- Wrong container port
- Firewall blocking port 3001

🧠 ENGINEERING REFLECTION

1. **Stateless services don't depend on prior data**
2. **Stateful services require session headers**
3. NAT shouldn't affect logical session identity

📄 LAB README.md (for repository)

```
None
# Lab 4 — Stateful vs Stateless Across Public & Private Subnets

## Architecture
```

```
Two LANs connected via a simulated internet (10.10.12.0/30)
- LAN A: 192.168.10.0/24
- LAN B: 192.168.20.0/24
Routers provide static routing + NAT.

## Steps

### 1. Setup Network
Configure R1 and R2:
- Static routes
- NAT overload
- Verify ping

### 2. Deploy IaC
Configure `ServerA.env` and `ServerB.env`
Deploy via VS Code, PowerShell or Bash using:
```

python manage.py deploy --profile week02-stateless-stateful

```
None

### 3. Validate
- Local stateless vs stateful
- Remote stateless vs stateful
- Test session persistence
- Capture NAT translations

## Troubleshooting
- Check interface status, NAT ACLs
- Verify client gateways

## Extensions
- OSPF instead of static routing
- Load balancer in front of APIs
```